JAVA MICRO-PROJECT · DEVELOPER REFERENCE GUIDE

Cine Java

Automated Movie Ticketing using Java Data Structures

A complete, section-by-section walkthrough of the console application: what every class does, why it was written that way, how the pieces interact, and the concepts you need to defend the project in a viva.

Language	Java (JDK 8 or newer, plain javac - no build tools)
Layers	Presentation (Main) · Service (BookingSystem, PaymentGateway) · Model (Movie, Show, Ticket...) · Exception
Data structures	ArrayList (catalogs), HashMap (seat map + user registry)
Core concept	synchronized booking to prevent double-booking under concurrency
Demo login	demo / demo123

Contents

1 Project overview - problem, objectives, scope
2 Architecture and file structure
3 Model layer - Movie, Theater, User, Ticket, Show
4 Exception layer - the four custom exceptions
5 Service layer - PaymentGateway and BookingSystem
6 Presentation layer - Main (console menus)
7 OOP concepts - where each one lives in the code
8 Collections Framework usage - ArrayList vs HashMap
9 Concurrency deep dive - how double-booking is prevented
10 Exception handling flow
11 Compiling, running, and the 2-minute demo script
12 Sample console output (annotated)
13 Viva preparation - 15 likely professor questions
14 Limitations and future enhancements

1. Project overview

1.1 The problem being solved

Manual cinema ticket counters suffer from three recurring failures: **(a)** long physical queues, **(b)** human error - most painfully assigning one seat to two different customers (double-booking), and **(c)** data corruption when several booking requests arrive at the same time. This project replaces that counter with an automated console application where validation and seat allocation are done by the program, deterministically and safely.

1.2 Objectives

- Model the ticketing domain - `Movie`, `Theater`, `Show`, `User`, `Ticket` - using OOP.
- Store and retrieve data efficiently with the Collections Framework: `ArrayList` for ordered catalogs, `HashMap` for O(1) seat-status and username lookups.
- Handle every foreseeable failure (invalid seat, taken seat, bad login, failed payment) with **custom exceptions** instead of crashing.
- Make seat booking **thread-safe**: when several threads race for the same seat, exactly one wins.
- Abstract payment behind an **interface** so the booking logic never depends on how payment actually happens.

1.3 Scope (deliberate decisions)

IN Console UI, in-memory data with mock seed data, login/register, seat layout display, multi-seat booking, simulated payment, concurrency demo. OUT Database persistence (JDBC/MySQL), GUI (JavaFX/Swing), real payment APIs - all listed as future enhancements (section 14).

> **One-line summary for the viva:** "It is a three-layer console application in plain Java where an `ArrayList` holds the catalogs, a `HashMap` holds each show's seat map, a `synchronized` method makes seat booking atomic, and four custom exceptions plus a `PaymentGateway` interface keep failures and payments cleanly separated from business logic."

2. Architecture and file structure

2.1 Three-layer architecture

Layer	Files	Responsibility
Presentation	`cine.Main`	Console menus, reading input, printing output. The ONLY class that talks to the user and the only one that touches `Scanner`.
Service	`cine.service.BookingSystem` `cine.service.PaymentGateway` `cine.service.MockPaymentGateway`	Business rules: registration, login, finding shows, running a booking transaction (seats then payment), ticket registry. Also the payment abstraction.
Model	`cine.model.Movie, Theater, User, Show, Ticket`	Domain entities + the seat map and its `synchronized` booking methods (the "business heart" lives in `Show`).
Exception	`cine.exception.*` (4 classes)	Meaningful, named failures the presentation layer can catch and translate into friendly messages.

Data flows strictly downward (UI calls service, service uses model); errors flow strictly upward as exceptions. No layer skips: `Main` never edits the seat map directly, and model classes never print to the console.

2.2 File tree

```
cine-java/
 ├─── run.sh                    - one-command compile+run (Linux/macOS)
 ├─── run.bat                   - one-command compile+run (Windows)
 ├─── README.md                 - quick-start + demo script
 └─── src/cine/
     ├─── Main.java               - console app entry point, menus, concurrency demo
     ├─── model/
     ├─    Movie.java           - catalog entry (id, title, genre, duration, price)
     ├─    Theater.java         - cinema hall (id, name, city)
     ├─    User.java            - account (username, password, fullName)
     ├─    Show.java            - screening + HashMap seat map + synchronized booking
     ├─    Ticket.java          - confirmed booking record (atomic id counter)
     ├─── exception/
     ├─    InvalidSeatException.java      - seat id not on the layout
     ├─    SeatUnavailableException.java  - seat already booked
     ├─    AuthenticationException.java   - bad login / taken username
     ├─    PaymentFailedException.java    - gateway refused/could not charge
     └─── service/
         ├─── PaymentGateway.java       - interface (abstraction)
         ├─── MockPaymentGateway.java   - simulated implementation
         └─── BookingSystem.java        - catalogs, users, booking transaction
```

3. Model layer

3.1 Movie - the catalog entry

Five `private final` fields (`id, title, genre, durationMins, price`) set once in the constructor and exposed only through getters - a textbook **immutable value object**. Because movies never change at runtime, nothing can put the catalog into an inconsistent state.

- `equals()` / `hashCode()` are overridden **on the id only**. This matters: `BookingSystem.getShowsFor(movie)` matches shows with `show.getMovie().equals(movie)`, and that comparison is reliable because of these overrides.
- `toString()` is overridden to produce a formatted catalog row (demonstrates `String.format` and method overriding).

3.2 Theater - composition

A simple value object (`id, name, city`). Each `Show` **has-a** `Theater` - a composition relationship you can point to when asked about object modeling. It exists so a ticket can print a real hall name instead of a bare id.

3.3 User - encapsulation of credentials

```
public class User {
    private final String username;
    private String password;
    private String fullName;

    /** Encapsulation: the password is never handed out, only verified. */
    public boolean checkPassword(String candidate) {
        return password != null && password.equals(candidate);
    }
}
```

The class has **no** **`getPassword()`** on purpose. Any code that needs to know whether a password is correct must go through `checkPassword()`. That is the cleanest one-line example of encapsulation in the project and a very good viva answer.

3.4 Ticket - the receipt

```
private static final AtomicInteger COUNTER = new AtomicInteger(1000);

public Ticket(User user, Show show, String seatId) {
    this.ticketId = "TKT-" + COUNTER.incrementAndGet();
    ...
}
```

- A ticket **links** a `User`, a `Show` and a seat id - it is created only inside `Show.bookSeat/bookSeats` after validation passes, so a Ticket can never exist for an invalid booking.
- Ids are generated by a `static AtomicInteger` (TKT-1001, TKT-1002, ...). It is thread-safe without any lock, which matters because tickets are created from racing threads in the concurrency demo.

- `asPrintable()` returns the boxed ticket layout you see after booking (movie, theater, show time, seat, customer, amount, transaction id, issue date).
- `price` is copied from the movie at booking time - a later price change cannot retroactively alter issued tickets.

3.5 Show - the heart of the project

```
public static final char[] ROWS = {'A','B','C','D','E'};   // 5 rows
public static final int COLS = 8;                          // 8 columns  -> 40 seats

private final Map<String, Boolean> seats = new HashMap<>();

public Show(...) {
    for (char row : ROWS)
        for (int col = 1; col <= COLS; col++)
            seats.put("" + row + col, false);      // every seat starts free
}
```

The seat map. Keys are seat ids ("A1" ... "E8"), values are `true` (booked) or `false` (free). Checking one seat is `seats.get(id)` - **O(1)** on average, versus scanning a whole 2-D array. Updating is a single `put`. This is exactly why the report chose a `HashMap`.

The double-booking guard (the most important method in the project)

```
public synchronized Ticket bookSeat(String seatId, User user)
        throws InvalidSeatException, SeatUnavailableException {
    String seat = seatId.trim().toUpperCase();
    Boolean booked = seats.get(seat);
    if (booked == null) throw new InvalidSeatException(seat);   // no such seat
    if (booked)         throw new SeatUnavailableException(seat); // taken
    seats.put(seat, true);                                      // claim it
    return new Ticket(user, this, seat);
}
```

Three things to memorize about this method:

1. **`synchronized` makes the check-then-act atomic.** Without it, two threads could both read `false` for seat E5, both pass the checks, and both write `true` - a classic **race condition** producing a double booking. With it, only one thread at a time holds the `Show` instance's **intrinsic lock (monitor)**, so the second thread is paused until the first has finished - and then it sees `true` and gets `SeatUnavailableException`.
2. **Validation order matters:** first "does the seat exist?" (`InvalidSeatException`), then "is it free?" (`SeatUnavailableException`).
3. **The method is the only way in.** The map is private; no caller can bypass validation by writing to `seats` directly.

Atomic multi-seat booking with rollback

```
public synchronized List<Ticket> bookSeats(List<String> seatIds, User user) {
    List<String> bookedSoFar = new ArrayList<>();
    try {
        for (String seat : seatIds) {
```

```
            ... same two checks ...
            seats.put(normalized, true);
            bookedSoFar.add(normalized);
        }
    } catch (InvalidSeatException | SeatUnavailableException e) {
        for (String seat : bookedSoFar)   // ROLLBACK: free everything
            seats.put(seat, false);       // taken in this call
        throw e;                          // all-or-nothing
    }
    ...build and return the tickets...
}
```

If a customer asks for "C5, C6, Z9", the first two would be claimed before the third fails. The rollback loop releases exactly the seats booked **in this call**, so the show returns to its previous state. Transactional behaviour, implemented in ~10 lines.

Supporting methods

- `releaseSeat(id)` - frees a seat; used by `BookingSystem` when payment fails, and by the demo to guarantee the race starts from a free seat.
- `availableCount()` - counts free seats (shown next to every show in the lists).
- `seatLayout()` - renders the console seating chart: a `[ SCREEN ]` header, column numbers, then one row letter + 8 cells, where `.` = available and `X` = booked.
- All of these are also `synchronized` - every read/write path through the seat map is protected, not just the happy path.

4. Exception layer

All four custom classes extend `Exception`, making them **checked exceptions**: the compiler forces every caller either to catch them or to declare them with `throws`. That is the right choice here, because each one models a *foreseeable business outcome* (a full show, a typo in a seat id), not a programming bug.

Exception	Thrown when	Thrown by	Extra data
`InvalidSeatException`	Seat id does not exist on the layout ("Z9").	`Show.bookSeat`, `bookSeats`, `BookingSystem.book` (unknown show id)	carries `getSeatId()`
`SeatUnavailableException`	Seat exists but is already booked - the double-booking attempt.	`Show.bookSeat`, `bookSeats`	carries `getSeatId()`
`AuthenticationException`	Login with wrong username/password; registering a taken username; booking while logged out.	`BookingSystem.login / register / book`	message only
`PaymentFailedException`	Gateway cannot complete the charge (e.g. empty mode).	`PaymentGateway` implementations; re-thrown by `BookingSystem.book` after seat rollback	message only

> **Design point worth stating in the viva:** each exception type is a different *category* of failure, so the UI can phrase a different message for each instead of one generic "something went wrong". Carrying the offending `seatId` inside the exception means the message does not need string parsing.

5. Service layer

5.1 PaymentGateway - abstraction and polymorphism

```
public interface PaymentGateway {
    String pay(Ticket ticket, String mode) throws PaymentFailedException;
}
```

One method, in, money out (a transaction id). `BookingSystem` holds a reference to the *interface*, never to `MockPaymentGateway` - the dependency is injected through `BookingSystem`'s constructor.
Consequences:

- The booking flow works with **any** future implementation (a real Razorpay/Stripe class, a "Pay at counter" class) without touching a line of booking code - **Open/Closed Principle**.
- `Main` chooses the implementation (`new MockPaymentGateway()`) - the wiring point of the whole app.

- This is runtime **polymorphism**: the same `gateway.pay(...)` call line dispatches to whichever implementation was injected.

5.2 MockPaymentGateway - the simulation

- Rejects a null/empty mode immediately with `PaymentFailedException` (validation lives with the implementer).
- `CASH` short-circuits to the transaction id `"CASH-AT-COUNTER"` - no bank involvement, by design.
- UPI/CARD: sleeps 200 ms to simulate network latency, then returns a random id like `TXN-188357` (via `String.format("TXN-%06d", ...)`).
- Correctly re-interrupts the thread on `InterruptedException` (`Thread.currentThread().interrupt()`) - standard best practice.

5.3 BookingSystem - the brain

```
private final List<Movie> movies = new ArrayList<>();       // ordered catalog
private final List<Show>  shows  = new ArrayList<>();       // ordered catalog
private final Map<String, User> users = new HashMap<>();    // username -> account
private final List<Ticket> ticketRegistry = new ArrayList<>();
private final PaymentGateway paymentGateway;                // injected abstraction
private User currentUser;                                   // session state
```

`seedMockData()` runs from the constructor, so the app is never empty: 3 movies (M1 Inception/Sci-Fi/Rs.250, M2 The Dark Knight/Action/Rs.220, M3 3 Idiots/Comedy/Rs.180), 2 theaters (T1 PVR Phoenix Mumbai, T2 INOX City Hub Pune), 5 shows (S1-S5), a few pre-booked seats (A1, A2, B3 on S1; C4 on S3 - booked by an internal "Front Desk" user so the seat chart is not all dots), and the demo account `demo / demo123`.

User management

- `register()` normalizes the username to lowercase and uses it as the HashMap key - `users.containsKey()` is an O(1) duplicate check; a taken name raises `AuthenticationException`.
- `login()` does one O(1) lookup, then `user.checkPassword()`. Success stores the user in `currentUser` - a tiny in-memory "session". `logout()` clears it.

The booking transaction - `book(showId, seatIds, paymentMode)`

This is the one method that coordinates the whole flow, and it is worth being able to recite:

1. **Auth guard** - no `currentUser`? Throw `AuthenticationException`.
2. **Resolve the show** - `findShow(showId)`, O(n) scan over a tiny list, unknown id reuses `InvalidSeatException`.
3. **Claim the seats** - one call to `show.bookSeats(...)`: atomic, all-or-nothing, exceptions on invalid/taken seats.
4. **Pay every ticket** - `gateway.pay(ticket, mode)`; the returned transaction id is stored on the ticket via `setPaymentTxnId`.
5. **Compensating action** - if payment throws, every seat of this booking is `releaseSeat`-ed and the failure is re-thrown. The customer is never left holding seats they did not pay for, and no orphan seats stay blocked.
6. **Record** - successful tickets go into `ticketRegistry`, which backs the "My bookings" menu.

Note for completeness: `getTicketsFor(user)` filters the registry with `ticket.getUser().equals(user)`. `User` does not override `equals`, so this compares references - which is correct here, because one session always carries the *same* `User` object that was passed into the booking. Overriding `equals` on `User` (on username) would be the first refactor if accounts were ever re-created between bookings.

6. Presentation layer - `Main`

6.1 Structure

`main()` builds the system (`new BookingSystem(new MockPaymentGateway())`), prints the banner (including the demo credentials), then loops forever between two menus: `showGuestMenu()` when logged out, `showMemberMenu()` when logged in. Every menu option maps to one small private method (`doLogin`, `doBrowseMovies`, `doBookTicket`, `doMyBookings`, `doConcurrencyDemo`, ...), each of which catches its own exceptions - one failing action never kills the loop.

6.2 Crash-proof input reading

```
private static int readInt(String prompt) {
    while (true) {
        System.out.print(prompt);
        String line = IN.nextLine();
        try {
            return Integer.parseInt(line.trim());
        } catch (NumberFormatException e) {
            System.out.println("  ! Please enter a number.");
        }
    }
}
```

Reading whole lines with `nextLine()` + parsing (instead of `nextInt()`) avoids the classic Scanner trap where a non-numeric token poisons the buffer and loops forever. Bad input is caught, a message is shown, and the same prompt is retried. Combined with the custom exceptions, **no input path in the app can crash it** - the "graceful error handling" evaluation criterion.

6.3 The booking screen (`doBookTicket`)

Sequence: `pickShow()` (numbered list + seats-free column) → `show.seatLayout()` chart → seats parsed from a comma-separated answer (e.g. `C5,C6`) → payment mode menu (1 UPI / 2 CARD / 3 CASH) → total = price × seat count → y/n confirmation → `system.book(...)` → print every ticket with `asPrintable()`.

The catch block uses Java 7 **multi-catch** for the two seat problems, and separate catches for authentication and payment - five failure modes, five clear messages, zero stack traces:

```
catch (InvalidSeatException | SeatUnavailableException e) {
    System.out.println("  ! Booking failed: " + e.getMessage());
} catch (AuthenticationException e) { ... } catch (PaymentFailedException e) { ... }
```

6.4 The thread-safety demo (`doConcurrencyDemo`)

```
final int THREADS = 5;
for (int i = 0; i < THREADS; i++) {
    final int racerId = i + 1;
    racers[i] = new Thread(() -> {
        try {
```

```
            Thread.sleep(50);                       // line everyone up first
            Ticket ticket = show.bookSeat(seat, new User(...));
            System.out.println("  [" + Thread.currentThread().getName() + "] SUCCESS -> " ...)
        } catch (SeatUnavailableException e) {
            System.out.println("  [...] rejected -> " + e.getMessage());
        } ...
    }, "Booker-" + racerId);
    racers[i].start();
}
for (Thread racer : racers) racer.join();           // wait for all outcomes
```

Five threads race to book the *same* seat (E5 on show S1, force-freed first). `Thread.sleep(50)` before the booking widens the window in which all five are inside the critical-section queue, so the race is visible; `join()` waits for every verdict before printing the summary line "Seat E5 booked = true (exactly one winner - no double booking)". Typical result: one `SUCCESS` and four `rejected`, in a different order every run - which is itself evidence of real concurrent scheduling.

7. OOP concepts - where each one lives

Concept	Where to point on the code
Encapsulation	All model fields are `private` with getters; `User` has no `getPassword()`, only `checkPassword()`; `Show`'s seat map is private and mutable only through its synchronized methods.
Inheritance	The four custom exceptions extend `Exception` (checked); they inherit `getMessage()` and the whole Throwable machinery. Everything implicitly extends `Object` - which is what gets overridden (`toString`, `equals`, `hashCode`).
Polymorphism	`BookingSystem` calls `gateway.pay(...)` through the interface; the same line runs `MockPaymentGateway` today, a real gateway tomorrow. Overridden `toString()` in Movie/Show/Ticket changes behaviour per class too.
Abstraction	`PaymentGateway` declares *what* payment means, not how; callers depend only on that contract. Layering itself (UI knows nothing of seat maps) is abstraction at the architecture level.
Multithreading	The 5-thread booking race in `Main.doConcurrencyDemo`; thread-safety from `synchronized` methods in `Show`; lock-free ids from `AtomicInteger` in `Ticket`.
Composition	`Show` has-a `Movie` and a `Theater`; `Ticket` has-a `User` and a `Show`.
Method overloading	`Show.bookSeat(String, User)` vs `bookSeats(List, User)` - same operation, single vs multiple seats.

8. Collections Framework usage

Structure	Field	Why this one
`ArrayList<Movie>`	movie catalog	Ordered, index-accessible, dynamic size; iteration order is stable so menus print the same way every time. Catalog is small and append-mostly - an array-backed list is the natural fit.
`ArrayList<Show>`	showtimes	Same reasoning; shows are listed, filtered (`getShowsFor`) and indexed positionally in the UI.
`HashMap<String,Boolean>`	seat map inside every `Show`	O(1) average `get` / `put` on seat ids; a missing key (`null`) cleanly distinguishes "not a seat" from "free seat"; direct update on booking - no grid scans.
`HashMap<String,User>`	user registry	Login is a single key lookup by (lowercased) username - O(1) instead of scanning a user list.
`ArrayList<Ticket>`	ticket registry	Append-only booking history; "My bookings" is a linear filter over it.

Viva nuance: `HashMap` is chosen over `Hashtable` (legacy, synchronized, slower) and over `TreeMap` (O(log n) sorting we do not need). Note the collections themselves are not thread-safe here - safety comes from `Show`'s `synchronized` methods guarding the seat map, which is a deliberate design: lock at the business-operation level, not the data-structure level.

9. Concurrency deep dive

9.1 The race, step by step (what would happen without `synchronized`)

Time	Thread Booker-1	Thread Booker-2	Seat map state
t1	`seats.get("E5")` → false		E5 free
t2		`seats.get("E5")` → false	E5 free (still!)
t3	checks pass → `put("E5", true)`		E5 booked by 1
t4		checks pass → `put("E5", true)`	**E5 "booked" twice** - two tickets, one seat

The failure comes from the gap between **check** (t1/t2) and **act** (t3/t4) - a *check-then-act* race condition. `synchronized bookSeat` closes the gap: the JVM guarantees a thread must acquire the `Show` instance's **monitor lock** before entering the method, holds it for the whole method body, and releases it on exit - so t2 cannot even begin until t3's write is visible.

9.2 The synchronization vocabulary (use these words in the viva)

- **Intrinsic lock / monitor:** every Java object has one; `synchronized` (instance method form) locks *this*. Here, each `Show` instance is its own lock - so booking seat E5 on S1 and booking D2 on S2 proceed fully in parallel (independent locks), while two requests for the same show serialize. Concurrency *within* a show is exactly as safe as it is *between* shows - the lock granularity is right.
- **Mutual exclusion:** the guarantee that only one thread executes the critical section at a time.
- **Visibility:** entering the monitor also guarantees the thread sees all writes made before the previous release - no stale `false` reads.
- **`AtomicInteger.incrementAndGet()`:** hardware-level compare-and-swap; a thread-safe counter with no lock at all - used for ticket ids.
- **`join()`:** blocks the caller until that thread terminates - used to collect all five verdicts before printing the summary.
- **Why the demo sleeps 50 ms:** so all threads are resident and queued before the first books; purely to make the demonstration reliable, never part of the correctness argument.

9.3 Transactionality beyond one seat

`bookSeats` holds the same lock across the entire multi-seat batch (both the claiming loop and the rollback loop), so even the rollback cannot interleave with another booking - the all-or-nothing guarantee holds under concurrency too. `BookingSystem.book` then adds the payment step: on `PaymentFailedException` it releases the seats (a *compensating action*, the same idea as a DB rollback, since we have no transaction manager).

10. Exception handling flow

```
user input: "book C5, C6, Z9 on S1, pay UPI"
   |
Main.doBookTicket()                    Presentation layer
   |  confirmation, mode chosen
   v
BookingSystem.book("S1", [C5,C6,Z9], "UPI")       Service layer
   |-- no currentUser? ----------------> AuthenticationException --> caught in Main: "! Please
   |-- findShow fails? ----------------> InvalidSeatException ------> caught in Main: "! Booki
   |-- Show.bookSeats()
   |      |-- "Z9" not a key ---------> InvalidSeatException --+  (seats claimed
   |      |-- seat already true ----> SeatUnavailableException |   in this call are
   |      |                                                    |  rolled back first)
   |      +-- all seats OK --> List<Ticket>
   |-- gateway.pay(...)  (MockPaymentGateway)
   |      |-- empty mode ------------> PaymentFailedException --> BookingSystem releases
   |      |                                                       seats, re-throws;
   '-- success --> tickets printed with asPrintable()           Main prints friendly msg
```

Two rules the flow follows: **(1)** exceptions are never swallowed - every catch either shows a message or re-throws; **(2)** the layer that can fix a problem is the one that catches it - `Show` restores state (rollback), `Main` talks to the human.

11. Compiling, running, demo script

11.1 Commands

```
# from the cine-java folder (any JDK 8+)
javac -d out $(find src -name "*.java")     # Windows: dir /s /b src\*.java > sources.txt
java -cp out cine.Main                      #          javac -d out @sources.txt
```

or double-click `run.bat` (Windows) / `bash run.sh` (Linux/macOS). `-d out` compiles into the `out/` folder preserving package folders; `-cp out cine.Main` puts `out` on the classpath and runs the entry class by its **fully qualified name** `cine.Main`.

11.2 The 2-minute demo script

1. **Login** (option 1) with `demo / demo123` - shows AuthenticationException if you mistype once.
2. **View movies & showtimes** (option 1) - catalog table, then 5 shows with seats-free counts.
3. **Book a ticket** (option 2): show S1 → layout (note pre-booked A1, A2, B3) → `C5,C6` → UPI → confirm → two boxed tickets.
4. **Error 1:** book seat `A1` → "already booked" (`SeatUnavailableException`), app continues.
5. **Error 2:** book seat `Z9` → "invalid seat" (`InvalidSeatException`), app continues.
6. **My bookings** (option 3) - lists tickets from the registry.
7. **Thread demo** (option 4) - one SUCCESS, four rejected, summary line. Close with option 0.

12. Sample console output (annotated)

```
================================================
    CINE JAVA - Automated Movie Ticketing
    (Collections + Exceptions + Threads)
================================================
  Demo login ->  username: demo   password: demo123

---- MAIN MENU (guest) ----
 1. Login  2. Register  3. View movies & showtimes
 4. Thread-safety demo  0. Exit
Choose an option: 1
Username: demo
Password: demo123
  Welcome back, Demo Student!

           [   S C R E E N   ]                    - Show.seatLayout()
     1 2 3 4 5 6 7 8
  A   X X . . . . . .                          - A1,A2 pre-booked
  B   . . X . . . . .                          - B3 pre-booked
  C   . . . . . . . .  ...

Enter seat(s) to book: C5
  Payment mode: 1 = UPI  2 = CARD  3 = CASH
  Total payable: Rs.250 for 1 seat(s)
Confirm booking? (y/n): y
```

```
  BOOKING SUCCESSFUL - 1 ticket(s)
+-------------------------------------------+
               CINE JAVA TICKET               - Ticket.asPrintable()
  Ticket   : TKT-1005                        - ids started at TKT-1001
  Movie    : Inception                         for the 4 seeded pre-bookings
  Theater  : PVR Phoenix, Mumbai
  Show     : Today 10:30 AM
  Seat     : E5
  Amount   : Rs.250  (paid: TXN-188357)       - MockPaymentGateway output
+-------------------------------------------+

Choose an option: 2   ... Enter seat(s): A1
  ! Booking failed: Seat 'A1' is already booked. Please pick another seat.

Choose an option: 2   ... Enter seat(s): Z9
  ! Booking failed: Invalid seat 'Z9'. Please choose a seat from the layout (e.g. C5).

---- THREAD-SAFETY DEMO ----
  5 threads will now try to book seat E5 on show S1 simultaneously...
  [Booker-2] rejected -> Seat 'E5' is already booked.    - order varies per run:
  [Booker-1] SUCCESS -> TKT-1006                         - evidence of real
  [Booker-5] rejected -> Seat 'E5' is already booked.       concurrent scheduling
  [Booker-4] rejected -> Seat 'E5' is already booked.
  [Booker-3] rejected -> Seat 'E5' is already booked.
  Seat E5 booked = true  (exactly one winner - no double booking)
```

13. Viva preparation - likely questions

1. Why a HashMap for seats instead of a 2-D array?

The natural state of a seat is a label ("C5") plus a flag. `HashMap` gives O(1) lookup and update on that label, and a `null` from `get()` cleanly distinguishes "not a seat" from "free seat" - no bounds checking code. A 2-D array would work but needs row/col arithmetic and index validation for every access.

2. What exactly does `synchronized` do here?

It wraps the method in the `Show` instance's intrinsic monitor lock: a thread must acquire the lock to enter, holds it for the whole body, and releases it at exit. Check-then-act on the seat map becomes atomic, and writes are visible to the next thread that acquires the same lock.

3. Which object is locked? Can two different shows book at the same time?

The `Show` instance itself (non-static `synchronized` locks `this`). Two bookings on *different* Show objects use different monitors and run fully in parallel - only requests for the *same* show serialize. The lock granularity matches the data's sharing granularity.

4. What is a race condition, concretely, in this project?

Two threads both reading `seats.get("E5") == false`, both passing validation, and both writing `true` - two tickets for one seat. See the timeline in section 9.1.

5. Why checked exceptions rather than RuntimeException?

Each failure is a foreseeable business outcome (full show, typo, wrong password), not a programming defect. Checked exceptions force the UI layer to handle them at compile time, so no failure path is forgotten. Runtime exceptions are reserved for bugs (like the NumberFormatException we still catch defensively in input parsing).

6. Difference between InvalidSeatException and SeatUnavailableException?

Input validity vs state conflict. "Z9" is not a seat at all (invalid input); "A1" exists but is taken (a conflict with another customer's earlier booking). Different causes, different user messages, different remedies.

7. Why is PaymentGateway an interface?

So booking logic depends only on the contract `pay(ticket, mode)`, not on any concrete gateway. We can inject MockPaymentGateway today and a real API class later with zero changes to BookingSystem - dependency inversion / Open-Closed Principle, and the polymorphic dispatch point of the app.

8. Why AtomicInteger for ticket ids? Why not a plain int counter?

Tickets are created from racing threads (the demo). `++counter` is three operations (read, add, write) and can lose updates. `AtomicInteger.incrementAndGet()` is a single lock-free atomic compare-and-swap, so ids are unique without synchronization.

9. What does bookSeats' rollback do? Is that a transaction?

It is all-or-nothing semantics: if any seat in the batch fails, the seats claimed earlier *in this call* are released again. It is the same idea as a database rollback, implemented manually because we have no transaction manager; BookingSystem does the same compensation when payment fails after seats were claimed.

10. What is join() and why is it in the demo?

`thread.join()` blocks the calling thread until that thread finishes. The demo needs all five verdicts printed and a stable final state before it prints the summary line, so it joins all five threads.

11. Why does the demo thread sleep 50 ms?

Only to make the race visible: it gives all five threads time to line up before any of them books. It has nothing to do with correctness - removing it still yields exactly one winner, just a less dramatic transcript.

12. Why ArrayList for the catalogs and not LinkedList?

Access is by position in printed menus and iteration dominates; ArrayList's O(1) get and compact memory layout fit that. LinkedList's O(1) insertions-in-the-middle are irrelevant for a small append-mostly catalog.

13. How does login work? Is it case-sensitive?

Usernames are stored as lowercase keys in a `HashMap<String,User>`; lookup is one O(1) call; the password is verified through `checkPassword()` without ever exposing it. Login is case-insensitive by the lowercase normalization; passwords are case-sensitive.

14. Where is the "session" stored? What happens on logout?

`BookingSystem.currentUser` - a single in-memory reference set at login, nulled at logout. Booking requires it, so the auth guard re-throws `AuthenticationException` after logout.

15. What are the current limitations?

Everything is in-memory (restart loses bookings), the UI is console-only, the payment gateway is a simulation, and accounts are not persisted. All four are intentional scope cuts; section 14 lists the upgrade paths.

14. Limitations & future enhancements

Limitation	Upgrade path (report's "future scope")
Data lost on exit (in-memory)	Persist movies/shows/bookings via JDBC + MySQL, replacing the seed method - collections become query results.
Console UI	JavaFX or Swing front-end reusing model + service layers unchanged (that is the payoff of layering).
Mock payment	Write a real `PaymentGateway` implementation calling a payments API - no other class changes.
Single session, no roles	Admin role for theater/show management; per-user session tokens.
Seat lock expires never	Optional: timed holds (release unpaid seats after N minutes) with a scheduler thread.

Cine Java - developer reference guide. Source of truth: the code in `src/cine/`; when in doubt, this document's code excerpts were taken verbatim from the shipped files.